NTNU
Department of Electronic Systems

TFE4141 Design of Digital Systems 1

Assignment 5: Finite State Machine design

Your team has been tasked with designing a component which reads values from a ready_valid bus, performs some computation on the data and writes the results to a different ready_valid bus. Your colleague has designed a general purpose datapath and you have been tasked with designing one of the possible controller units for it. The component is to receive two unsigned values through a ready_valid bus and output the absolute value of the difference between the two. This is described in Equation 1. The ready_valid interface is the same as in Assignment 4.

$$output_N = | input_{2N+1} - input_{2N} | \quad (1)$$

Some of your other colleagues have implemented controller units that implement different algorithms (max_of_3 and sum_of_n). They have also documented their process so that you may learn from their work (DDS1_assignment_5_examples.pdf).

Preparation

You should read this document to the end before starting to work on the assignment. Also check for additional information that may be posted on Blackboard and Piazza.

1. Download assignment_5.zip and extract it to some known location.
2. Open Vivado.
3. Click 'Tools -> Run tcl script' and navigate to the location you extracted the zip file and run the script regenerate_projects.tcl. Three new projects (assignment_5, max_of_3 and sum_of_n) should now appear in the Recent projects" tab.
4. If the projects for some reason do not appear, or are deleted from the list later, you can select "Open Project" in the Vivado "Quick Start" menu and navigate to and open the xpr file of the assignment_5 project (typically placed at /assignment_5/assignment_5/assignment_5.xpr). The same procedure can be used for projects max_of_3 and sum_of_n.

Note that you only have to modify the assignment_5 project, but you are strongly encouraged to look at the others for inspiration.

Task 1

Study the description of the hardware below. Write psuedocode (or a real language like C or Python if you prefer) that uses this hardware to achieve the absolute value algorithm described above.

Task 2

Convert the psuedocode to a finite state machine and draw the state diagram for it. Describe all the transitions with the associated conditions and actions. You may choose to implement the state machine as a Mealy or a Moore state machine.

Task 3

Implement the designed finite state machine in the controller module using vhdl. Here you should be careful with unintended latches. Test the design by running a simulation with the provided testbench.

Task 4

Run synthesis and implementation on the whole project. Does the design pass the testbench in post synthesis simulation? What about post implementation simulation?

The hardware

The datapath roughly implements the following actions:

1. Read a value from the input bus and store in a register.
2. Compare values in two different registers.
3. Perform an arithmetic operations between two registers and store the result.
4. Write the results to the output bus.

The ALU implements the opcodes as described in Table 2. For all opcodes, the ALU also outputs the comparisons of the inputs. *Input_equals* goes high when *Input_A* = *Input_B* and *Input_greater* goes high when *Input_A* > *Input_B*.

Mnemonic	Encoding		Description
load	0_{10}	000_2	Pass input A to the output
add	1_{10}	001_2	Add input A to input B
sub	2_{10}	010_2	Subtract input B from input A
and	3_{10}	011_2	Bitwise AND between input A and input B
or	4_{10}	100_2	Bitwise OR between input A and input B
xor	5_{10}	101_2	Bitwise XOR between input A and input B
shift_left	6_{10}	110_2	Input A shifted left B times
shift_right	7_{10}	111_2	Input A shifted right B times

Table 1: The opcodes implemented in the ALU

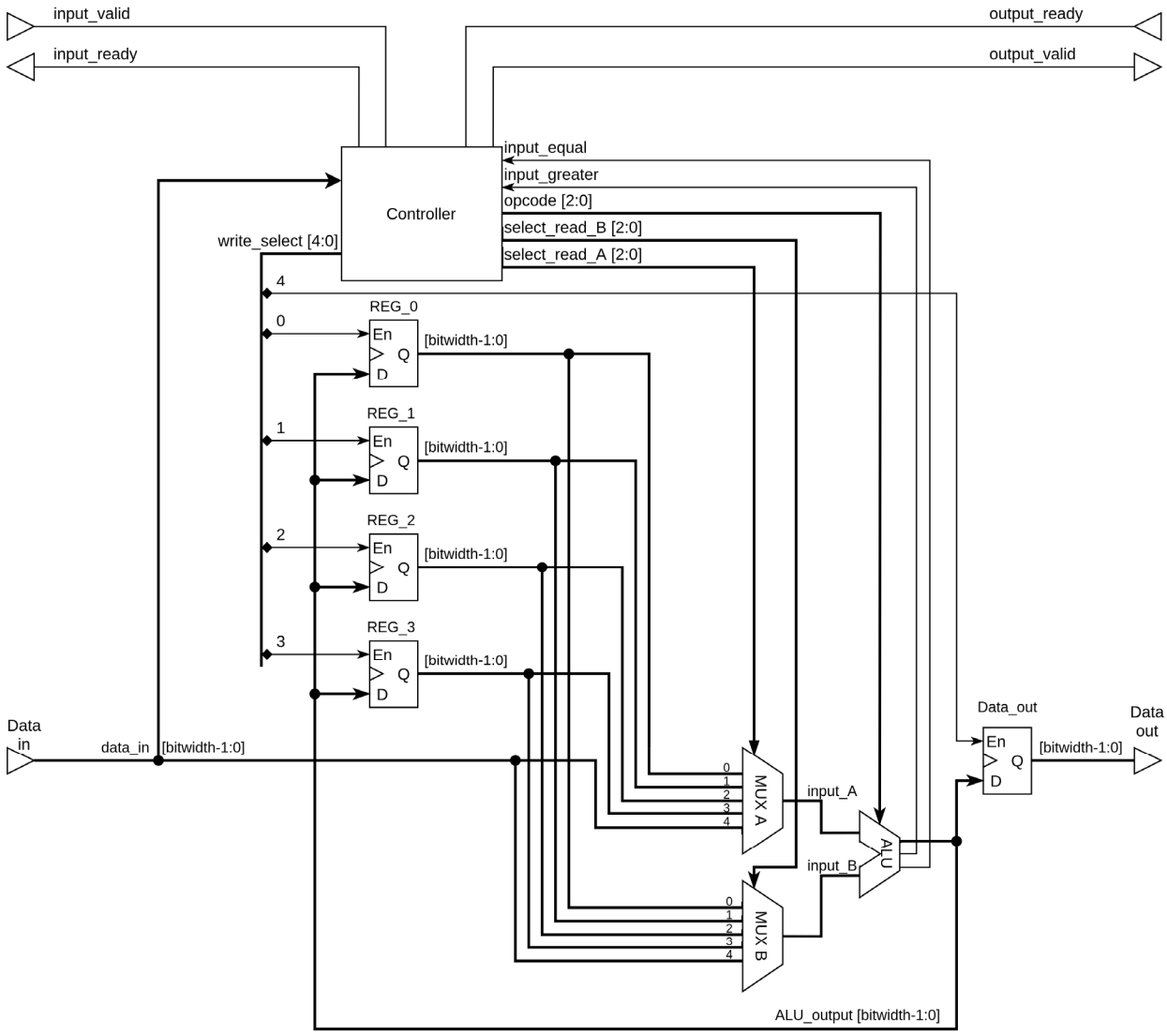

Figure 1: The datapath and controller unit